

AI-Powered Multi-Agent Code Reviewer

Project Report

Project Report on

Multi-Agent LLM-Driven Code Review System

Course: Special Topics (CMPE- 297)

MS Software Engineering (Fall 2025)

Submitted by:

Aditya Rajpurohit (Student ID: 018211414)

Rutuja Bhaskarrao Patil (Student ID: 018233098)

Shruti Goyal (Student ID: 017556682)

Manjunatha Inti (Student ID: 018192187)

Guided by:

Prof. Vijay Eranti

Academic Year: 2025

ABSTRACT

Modern software development relies heavily on code reviews to ensure correctness, security, and maintainability. However, traditional static analysis tools provide limited semantic understanding, while human review processes are time-consuming and inconsistent. To address these challenges, this project presents an AI-powered multi-agent code review system that performs automated code analysis, bug detection, security auditing, fix generation, and GitHub pull request (PR) creation within a unified, production-inspired workflow.

The system supports three input modes: direct code editing, local file uploads, and GitHub repository integration. A structured multi-agent orchestration pipeline coordinates specialized agents including Static Analysis, Deep Review, Security, Bug Detection, Fix Generation, and Evaluation agents to analyze source code from multiple perspectives. Each agent contributes focused insights, enabling comprehensive and explainable reviews. Large Language Models (LLMs) are used to reason about code semantics, identify vulnerabilities, and generate corrected implementations.

The platform integrates tightly with GitHub OAuth, allowing authenticated users to browse repositories, review files, and automatically generate pull requests with AI-suggested fixes. To ensure reproducibility, observability, and scalability, the system incorporates a cloud-native MLOps stack, persisting in review metadata in PostgreSQL, storing fixed code artifacts in Amazon S3, and logging agent execution details to CloudWatch.

Experimental results demonstrate that the multi-agent approach consistently identifies security risks, logic bugs, and style violations while producing high-quality fixes and evaluation summaries. Overall, this project illustrates how LLM-driven multi-agent systems, combined with cloud infrastructure and developer tooling, can significantly enhance modern software engineering workflows through automated, auditable, and scalable code review processes.

1. INTRODUCTION

Code review is a critical practice in modern software engineering, serving as a primary mechanism for identifying bugs, enforcing coding standards, and improving overall software quality. Despite its importance, traditional code review workflows remain largely manual, time-consuming, and dependent on individual expertise. While static analysis tools and linters can detect syntactic issues and known anti-patterns, they lack deeper semantic understanding and are unable to propose meaningful, context-aware fixes. Recent advances in Large Language Models (LLMs) have demonstrated strong capabilities in reasoning about source code, identifying logical errors, and generating high-quality refactoring's. However, most existing AI-assisted coding tools operate as single-pass systems that provide unstructured feedback, limited traceability, and minimal integration with real-world development workflows such as GitHub-based collaboration and pull request management.

This project addresses these limitations by designing and implementing an AI-powered multi-agent code review system that combines LLM reasoning with structured agent orchestration, cloud-native infrastructure, and developer-facing tooling. Instead of relying on a monolithic reviewer, the system decomposes the review task into multiple specialized agents—each responsible for a distinct analytical perspective, such as static checks, semantic review, security analysis, bug detection, fix generation, and post-fix evaluation. This modular approach improves coverage, interpretability, and extensibility.

The system supports multiple code ingestion modes, including direct code editing, local file uploads, and GitHub repository browsing through OAuth-based authentication. For GitHub-based workflows, the platform enables users to review repository files and automatically generate pull requests containing AI-proposed fixes. All review sessions, agent outputs, execution metadata, and generated artifacts are tracked through a cloud-based MLOps stack, ensuring reproducibility and observability.

By integrating multi-agent LLM reasoning, GitHub automation, and cloud MLOps best practices, this project demonstrates a practical and scalable approach to intelligent code review. The resulting system reflects a production-inspired architecture that bridges research advances in agentic AI with real-world software engineering requirements.

2. RELATED WORK

Research and tooling related to automated code review span multiple domains, including static analysis, AI-assisted programming, multi-agent reasoning systems, and MLOps-enabled AI pipelines. This section reviews relevant prior work and highlights how the proposed system advances the state of the art.

2.1 Traditional Static Analysis Tools

Static analysis tools such as ESLint, Flake8, Bandit, and SonarQube are widely used to identify syntactic errors, code smells, and known security vulnerabilities. These tools rely on predefined rules, abstract syntax trees (ASTs), or pattern matching to flag issues. While effective for enforcing coding standards and detecting common mistakes, they lack semantic reasoning and are unable to understand developer intent, cross-line logic, or higher-level architectural concerns. Moreover, traditional static analyzers typically do not generate corrected code or evaluate the quality of proposed fixes.

2.2 LLM-Based Code Assistants

Recent LLM-powered tools such as GitHub Copilot, Amazon CodeWhisperer, and ChatGPT-based coding assistants demonstrate strong performance in code completion, refactoring, and natural-language-driven code synthesis. However, these systems primarily function as single-agent, single-pass assistants. Feedback is often unstructured, difficult to audit, and not integrated into an end-to-end review workflow. Additionally, most tools focus on generation rather than systematic review, security auditing, or fix evaluation, and provide limited support for automated pull request creation and traceability.

2.3 Automated Program Repair and Review Systems

Prior research in automated program repair (APR) explores rule-based and learning-based approaches to fixing software defects. While some systems generate patches, they often require test suites or labeled datasets and are constrained to narrow bug classes. In contrast, LLM-based approaches remove the dependency on explicit supervision but require careful

orchestration to ensure reliability, coverage, and safety—challenges not addressed by earlier APR systems.

2.4 Multi-Agent Reasoning Frameworks

Multi-agent AI systems such as AutoGPT, SWE-Agent, and Devin demonstrate the effectiveness of decomposing complex tasks into specialized agents that interact through structured workflows. These systems show promise in software engineering tasks but often lack production-grade observability, evaluation, and cloud integration. Many remain research prototypes without systematic logging, artifact persistence, or integration into developer platforms like GitHub.

2.5 MLOps and Observability in AI Systems

Modern MLOps frameworks emphasize reproducibility, artifact management, evaluation tracking, and centralized logging. Best practices outlined in recent literature advocate storing model outputs, execution metadata, and evaluation results to support auditing and iterative improvement. However, few AI-powered code review systems integrate these principles end-to-end.

The proposed system distinguishes itself by combining multi-agent LLM reasoning with real-world software engineering workflows. Unlike traditional static analyzers or single-agent LLM tools, this project integrates specialized review agents, automated fix generation, evaluation scoring, GitHub PR automation, and a cloud-native MLOps backbone. This holistic design bridges the gap between research-oriented agent frameworks and practical, production-ready AI systems for code review.

3. DATA

Unlike conventional machine learning projects that rely on static, pre-collected datasets, this system operates on dynamic, user-provided source code that is ingested and processed at runtime. The primary “data” in this project consists of software artifacts and metadata generated during the code review lifecycle, rather than labeled training examples.

3.1 Input Data Sources

The system supports three modes of data ingestion, enabling flexibility across different development workflows:

- **Editor Input:** Users may directly paste or type source code into the web-based editor. This mode is intended for rapid experimentation, small snippets, and educational use cases.
- **File Uploads:** Users can upload local source files, which are analyzed and optionally rewritten by the system. This mode supports realistic single-file review scenarios.
- **GitHub Repository Integration:** Through OAuth-based authentication, users can provide a GitHub repository URL, branch, and file path. The system dynamically fetches the selected file for review and can later generate a pull request containing AI-proposed fixes.

Each review session is associated with contextual metadata, including the source type, repository URL (if applicable), file path, branch name, user identifier, and timestamps.

3.2 Generated Review Data

As code flows through the multi-agent pipeline, the system generates multiple forms of structured and unstructured data:

- **Agent-specific review comments**, categorized by severity and type (e.g., security, bug, style, information)
- **Aggregated review summaries**, produced by merging and de-duplicating agent outputs

- **Fixed code artifacts**, generated by the Fix Agent as a fully rewritten version of the original file
- **Evaluation outputs**, including qualitative summaries, numeric scores, and risk-level assessments

These outputs collectively represent the system's reasoning process and form the basis for automated pull request generation and evaluation.

3.3 Privacy-Aware Data Handling

To minimize exposure of potentially sensitive source code, the system adopts a privacy-conscious data strategy. Full raw source files are not stored directly in the database. Instead, the system persists:

- **Cryptographic hashes** of the original code
- **Short preview snippets** limited to a fixed character budget
- **References (keys)** to generated artifacts stored externally

This design allows the system to support reproducibility and traceability without retaining full proprietary code in long-term storage.

3.4 Data Storage and Persistence

All generated data and metadata persisted using a cloud-native MLOps architecture:

- **Amazon S3** stores fixed code artifacts and structured outputs using a hierarchical, session-based directory structure.
- **PostgreSQL (AWS RDS)** stores structured metadata, including review session records, agent outputs, evaluation scores, execution timestamps, and artifact references.
- **AWS CloudWatch Logs** capture detailed agent-level execution traces, enabling observability, debugging, and performance analysis.

Together, these components provide a scalable and auditable data layer that supports monitoring, evaluation, and future system extensions.

4. METHODOLOGY

4.1 Multi-Agent Architecture

At the heart of the system is a multi-agent architecture designed to decompose the complex task of code review into smaller, specialized reasoning units. Instead of relying on a single monolithic LLM call, the system orchestrates multiple agents, each responsible for analyzing the code from a distinct perspective. This design improves coverage, interpretability, and reliability of the review process.

Each agent operates independently, receives the same code context, and produces structured outputs that are later merged by the orchestrator. All agents are implemented using LLM-backed reasoning prompts and follow a standardized input–output contract to ensure consistency.

4.1.1 Static Analysis Agent

The Static Analysis Agent performs lightweight, rule-based inspection of the source code before invoking deeper LLM reasoning. Its responsibilities include:

- Identifying TODO comments, dead code, and unused variables
- Detecting basic formatting inconsistencies
- Flagging obvious anti-patterns and suspicious constructs

This agent establishes a baseline set of issues and provides fast feedback with minimal latency. Its output also serves as contextual signals for downstream agents.

4.1.2 Deep Review Agent

The Deep Review Agent is an LLM-powered component responsible for semantic and stylistic analysis of the code. It examines:

- Logical correctness and intent alignment
- Readability, maintainability, and structure
- Idiomatic usage of the target programming language
- Adherence to best practices and conventions

Unlike static analyzers, this agent reasons about the *meaning* of the code rather than surface-level patterns, allowing it to identify subtle issues and refactoring opportunities.

4.1.3 Security Analysis Agent

The **Security Agent** focuses specifically on identifying vulnerabilities and unsafe coding practices. It evaluates the code for:

- Injection risks (e.g., SQL injection, command injection)
- Hardcoded secrets or credentials
- Improper input validation or sanitization
- Unsafe logging of sensitive data
- Missing authorization or access control checks

Security findings are assigned higher priority during aggregation to ensure critical risks are addressed first.

4.1.4 Bug Detection Agent

The Bug Detection Agent performs targeted reasoning over potential runtime and logic errors, including:

- Edge-case failures
- Incorrect control flow
- Unhandled exceptions
- Race conditions or async misuse (for supported languages)
- Resource leaks or improper cleanup

This agent complements the Deep Review Agent by focusing on functional correctness and execution behavior.

4.1.5 Fix Generation Agent

The Fix Agent synthesizes a corrected version of the original source file by incorporating feedback from all prior agents. Rather than generating isolated

patches, it produces a fully rewritten, clean version of the file, ensuring consistency and readability.

Its responsibilities include:

- Resolving detected bugs and vulnerabilities
- Improving code structure and clarity
- Applying best practices and safer patterns
- Preserving original functionality and intent

The generated output is treated as a candidate fix for automated pull request creation.

4.1.6 Evaluation Agent

The Evaluation Agent performs post-hoc assessment of the generated fix. It compares the original code, review comments, and fixed version to produce:

- Quantitative scores (correctness, security, style)
- Qualitative summaries of improvements
- A risk-level assessment indicating residual concerns

This agent provides an explicit feedback loop, improving transparency and enabling downstream evaluation and reporting.

4.2 Orchestrator Workflow

The multi-agent review process is coordinated by a centralized **orchestrator** that manages execution order, data flow, aggregation, and persistence. The orchestrator serves as the control plane of the system, ensuring that each agent operates with the appropriate context and that outputs are combined into a coherent final result.

4.2.1 Session Initialization

Each review begins with the creation of a **review session**, which uniquely identifies the execution. The orchestrator records session metadata, including:

- Source type (editor, file upload, or GitHub)
- Repository information and file path (if applicable)

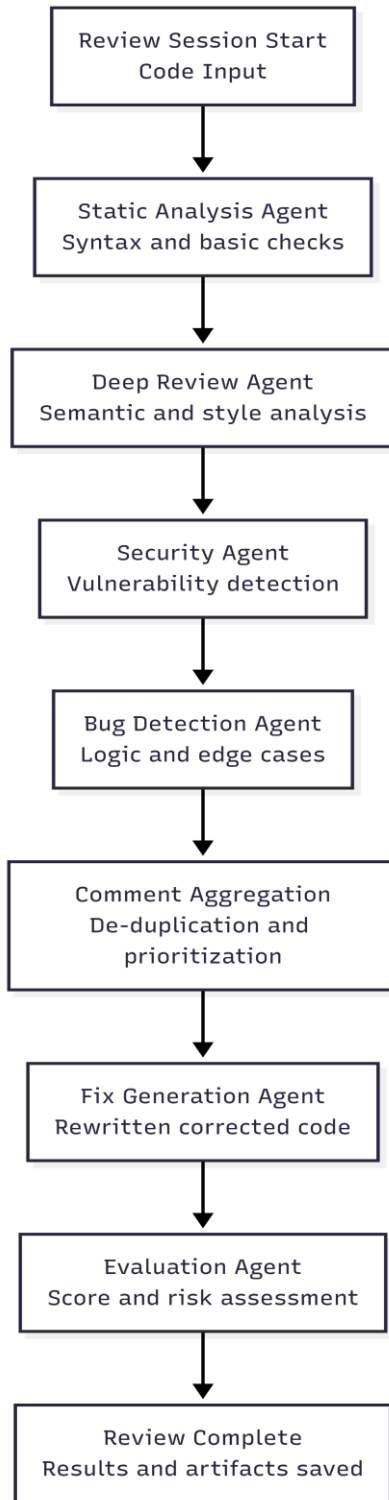


Figure 4.2: Multi-agent code review pipeline illustrating sequential analysis, fix generation, and evaluation

- User identifier
- Timestamp and execution parameters

This metadata enables traceability across agents and downstream artifacts.

4.2.2 Sequential Agent Execution

Agents are executed in a **deterministic sequence**, reflecting increasing analytical depth:

1. **Static Analysis Agent** performs lightweight rule-based checks and establishes baseline issues.
2. **Deep Review Agent** conducts semantic and stylistic reasoning over the code.
3. **Security Analysis Agent** evaluates potential vulnerabilities and unsafe patterns.
4. **Bug Detection Agent** identifies logic errors, edge cases, and runtime risks.

Each agent receives the same source code along with accumulated contextual signals from previous agents. Execution time and outputs are recorded for observability.

4.2.3 Comment Aggregation and Prioritization

Agent outputs are merged by the orchestrator into a unified review report. During this step:

- Duplicate or overlapping comments are de-duplicated
- Findings are categorized (bug, security, style, informational)
- Issues are prioritized, with security and correctness concerns ranked highest

This aggregation step ensures clarity and avoids overwhelming the user with redundant feedback.

4.2.4 Fix Generation

Once review comments are finalized, the **Fix Generation Agent** is invoked. The orchestrator supplies:

- Original source code

- Aggregated and prioritized review comments
- Language and context constraints

The Fix Agent produces a complete, corrected version of the file, which represents the system's proposed solution.

4.2.5 Post-Fix Evaluation

The generated fix is then passed to the **Evaluation Agent**, which assesses the quality and safety of the changes. The evaluation includes:

- Scoring across multiple dimensions (correctness, security, style)
- Identification of remaining risks or unresolved issues
- Generation of a concise summary explaining improvements

These results provide transparency and support both automated and human decision-making.

4.2.6 Persistence and Observability

After completion, the orchestrator persists all relevant data:

- **Metadata and agent outputs** are stored in PostgreSQL
- **Fixed code artifacts** are uploaded to Amazon S3
- **Agent-level logs and timings** are written to CloudWatch

This design enables reproducibility, auditing, performance monitoring, and future analysis.

4.2.7 GitHub Integration and PR Creation

For GitHub-based workflows, the orchestrator supports authenticated pull request creation. Using OAuth tokens, the system:

- Commits the fixed code to a new branch
- Creates a pull request with AI-generated descriptions
- Records PR metadata for tracking and evaluation

5. SYSTEM ARCHITECTURE

The system is designed as a modular, cloud-native full-stack application that integrates a web-based user interface, a backend orchestration service, and multiple cloud services for persistence, observability, and scalability. The architecture follows separation of concerns, enabling independent evolution of the frontend, backend, and infrastructure components.

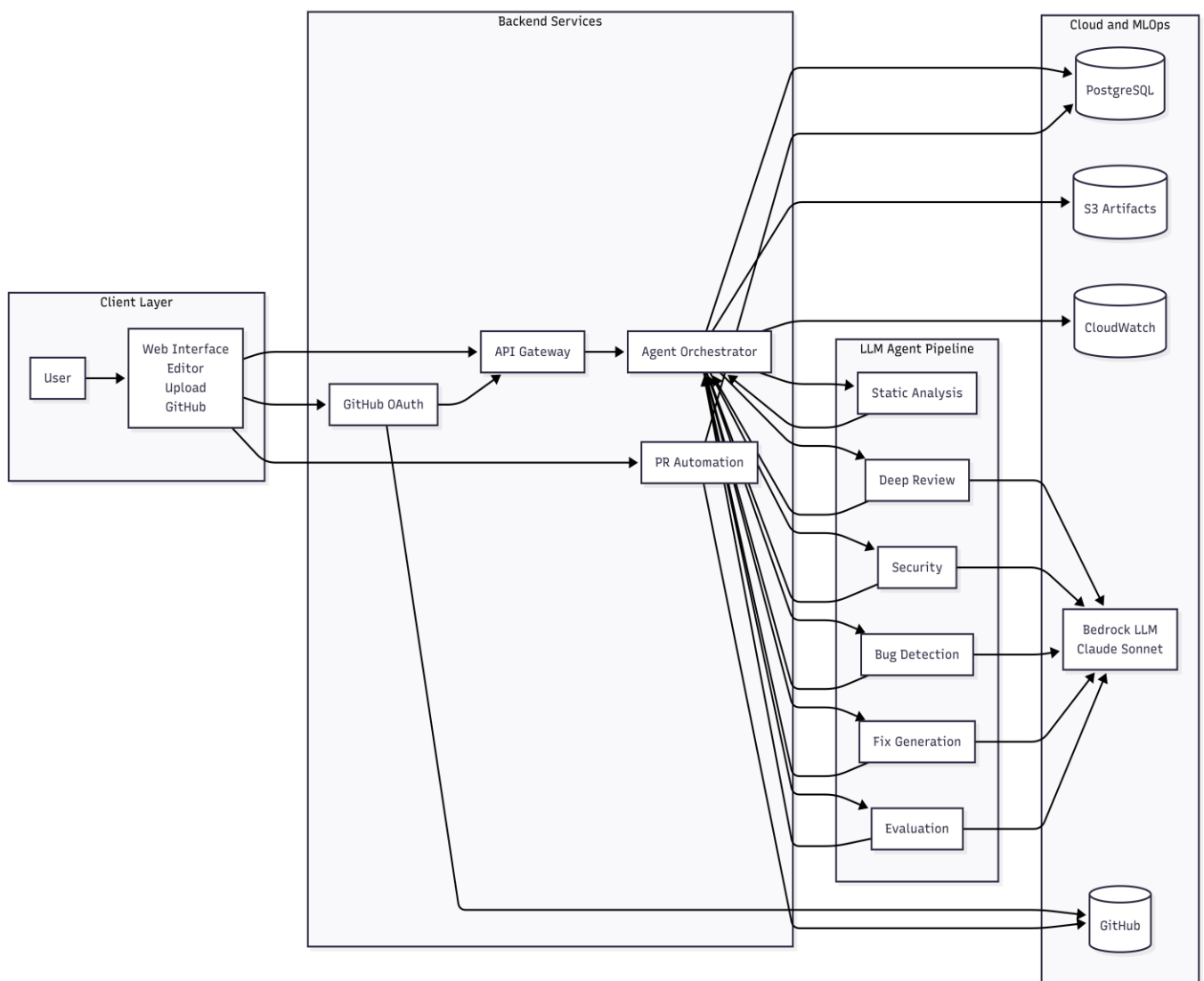


Figure 5.1: Canva-style system architecture of the AI-powered multi-agent code reviewer

5.1 Frontend Architecture

The frontend is implemented using **Next.js**, providing a responsive and interactive user experience for initiating and managing code reviews. It serves as the primary interface through which users interact with the multi-agent system.

Key Features

- **Multiple Review Modes**
 - *Editor Mode*: Paste and review code snippets directly.
 - *File Upload Mode*: Upload local source files for review and fix generation.
 - *GitHub Mode*: Browse repository files after OAuth authentication and select files for review.
- **Fix Visualization**
 - Side-by-side comparison of original and fixed code.
 - Display of aggregated review comments and evaluation summaries.
- **Pull Request Interface**
 - Option to generate pull requests directly from the UI for GitHub-based reviews.

The frontend communicates exclusively with backend API endpoints, ensuring a clean separation between presentation logic and core review functionality.

5.2 Backend Architecture

The backend is implemented using Node.js with TypeScript and Express, and acts as the control layer of the system. It exposes RESTful APIs that coordinate code ingestion, agent execution, fix generation, evaluation, and persistence.

Core Responsibilities

- Input validation and preprocessing

- Orchestration of the multi-agent pipeline
- Interaction with LLM providers
- Artifact storage and metadata persistence
- GitHub OAuth handling and PR creation

Key API Endpoints

Endpoint	Description
/api/review	Review code submitted via editor
/api/review/file	Review uploaded files
/api/review/github	Review GitHub repository files
/api/github/tree	Fetch repository file structure
/api/review/github/pr	Create pull request with generated fix
/auth/github/login	Initiate GitHub OAuth login
/auth/github/callback	Handle OAuth callback and token exchange

5.3 Cloud Infrastructure and MLOps Components

The system integrates several cloud services to support persistence, observability, and scalability, following modern MLOps best practices.

5.3.1 PostgreSQL (AWS RDS)

A managed PostgreSQL database stores structured metadata for each review session, including:

- Review session identifiers

- Agent outputs and execution timings
- Evaluation scores and summaries
- Pull request metadata
- References to stored artifacts

This enables historical analysis, auditing, and reproducibility.

5.3.2 Amazon S3

Amazon S3 is used for persistent storage of generated artifacts, particularly fixed code outputs. Artifacts are organized using a session-based directory structure:

fixed/<source>/<session-id>/<filename>

This design supports efficient retrieval, isolation between sessions, and long-term storage.

5.3.3 AWS CloudWatch Logs

CloudWatch Logs capture detailed execution traces for each agent, including:

- Start and end timestamps
- Execution duration
- Comment counts and evaluation metrics

These logs enable performance monitoring, debugging, and observability across the multi-agent pipeline.

5.3.4 LLM Provider Integration

The backend integrates with Amazon Bedrock, using Claude-based LLMs for agent reasoning. The architecture is provider-agnostic, allowing future integration with alternative LLM providers without changes to the orchestration logic.

6. EXPERIMENTS & RESULTS

6.1 Experimental Setup

To evaluate the effectiveness of the proposed multi-agent code review system, we conducted a series of controlled experiments using representative source code samples and real-world repository workflows. The goal of these experiments was to assess the system's ability to detect issues, generate meaningful fixes, and integrate seamlessly with developer workflows.

Input Modalities

The system was tested using all three supported input modes:

- **Editor Mode:** Source code snippets pasted directly into the web interface.
- **File Upload Mode:** Local source files uploaded for review and fix generation.
- **GitHub Integration Mode:** Source files fetched from GitHub repositories using OAuth-based authentication, followed by optional pull request creation.

These modes allowed evaluation across both synthetic examples and realistic development scenarios.

Programming Languages

Experiments primarily focused on widely used programming languages, including:

- **Python** for testing security vulnerabilities, database access patterns, and error handling.
- **JavaScript** for testing asynchronous logic, promise handling, and runtime bugs.

These languages were chosen due to their prevalence in modern software systems and their susceptibility to common security and logic errors.

Execution Environment

The system was deployed in a local development environment with cloud service integration:

- **Frontend:** Next.js web application accessed via a browser.

- **Backend:** Node.js and TypeScript–based API server orchestrating agent execution.
- **LLM Provider:** Amazon Bedrock using Claude Sonnet models for all LLM-backed agents.
- **Persistence and Logging:** PostgreSQL for metadata storage, Amazon S3 for artifact storage, and CloudWatch Logs for agent-level execution tracing.

Each experiment was executed within an isolated review session to ensure reproducibility and accurate tracking of agent outputs and metrics.

Evaluation Procedure

For each input case, the following steps were performed:

1. Submit source code through one of the supported input modes.
2. Execute the full multi-agent pipeline, including review, fix generation, and evaluation.
3. Collect agent outputs, generated fixes, evaluation scores, and execution metadata.
4. Analyze results qualitatively and quantitatively.

This setup enabled a comprehensive assessment of both system functionality and review quality under realistic usage conditions.

6.2 Test Scenarios

To demonstrate the capabilities of the multi-agent code review system, we evaluated it across multiple representative scenarios that reflect common issues encountered in real-world software development. Each scenario was selected to exercise different aspects of the agent pipeline, including security analysis, bug detection, fix generation, and GitHub workflow integration.

6.2.1 Security-Vulnerable Python Code

Scenario Description:

A Python source file containing database access logic with unsafe string

concatenation, insufficient input validation, and weak error handling was submitted through the editor interface.

Expected Behavior:

The system was expected to detect security vulnerabilities, identify logic flaws, and propose a safer implementation.

Observed Agent Behavior:

- The **Security Agent** flagged SQL injection risks due to unparameterized queries and highlighted unsafe logging of sensitive information.
- The **Bug Detection Agent** identified potential runtime failures, including division-by-zero risks and improper exception handling.
- The **Deep Review Agent** noted style inconsistencies and recommended clearer function structure.
- The **Static Analysis Agent** detected TODO markers and unused variables.

Outcome:

The Fix Generation Agent produced a revised version of the file using parameterized queries, improved exception handling, and safer logging practices. The Evaluation Agent assessed the fix as significantly reducing security risk while preserving original functionality.

6.2.2 JavaScript Asynchronous Logic Errors

Scenario Description:

A JavaScript file containing asynchronous operations with missing await statements and unhandled promise rejections was tested using the file upload mode.

Expected Behavior:

The system was expected to identify logical errors related to asynchronous execution and recommend corrections.

Observed Agent Behavior:

- The **Bug Detection Agent** detected missing await keywords and unhandled promise chains.
- The **Deep Review Agent** identified readability and maintainability issues related to nested asynchronous logic.
- The **Security Agent** flagged insufficient error handling in asynchronous flows.

Outcome:

The Fix Generation Agent generated a corrected version of the code with proper async/await usage and explicit error handling, resulting in more predictable runtime behavior.

6.2.3 GitHub Repository Review and Pull Request Creation

Scenario Description:

A GitHub repository was accessed using OAuth authentication. A selected file from a target branch was reviewed, and an automated pull request was generated with the proposed fix.

Expected Behavior:

The system was expected to seamlessly integrate with GitHub, review the selected file, and create a valid pull request under the authenticated user's account.

Observed Agent Behavior:

- The system successfully fetched repository structure and file contents.
- The full multi-agent review pipeline executed on the selected file.
- The Fix Generation Agent produced a corrected version suitable for commit.
- The Pull Request Automation module created a new branch and opened a pull request containing the fix and AI-generated summary.

Outcome:

The pull request was successfully created and logged, demonstrating end-to-end integration between AI review, code generation, and developer collaboration workflows.

6.3 Qualitative Results

This section presents qualitative observations from the experimental runs, focusing on the clarity, usefulness, and consistency of the outputs generated by the multi-agent pipeline. Screenshots from the web interface, agent responses, and GitHub pull request creation are used to illustrate system behavior.

6.3.1 Review Output and Agent Feedback

For each reviewed file, the system displayed a consolidated set of review comments in the user interface. These comments were clearly grouped and labeled based on issue type, including security concerns, logic bugs, and stylistic recommendations.

The Security Agent outputs were particularly precise, highlighting high-risk patterns such as unsafe database queries, hardcoded credentials, and insufficient input validation. These findings were consistently prioritized above stylistic or readability issues, demonstrating effective severity-based ordering during aggregation.

The **Bug Detection and Deep Review Agents** provided complementary feedback by identifying logical flaws, edge cases, and maintainability issues. Importantly, the feedback was expressed in natural language that was easy to interpret, making the results accessible even to users without deep security expertise.

6.3.2 Fix Generation and Code Improvement

Following the review stage, the system generated a fully rewritten version of the original source file. The Fix Generation Agent incorporated feedback from all preceding agents into a single, coherent implementation rather than producing fragmented patches.

Qualitatively, the generated fixes demonstrated:

- Improved safety through parameterized queries and defensive coding practices
- Clearer control flow and error handling
- More consistent naming conventions and formatting

- Removal of unused variables and TODO comments

The side-by-side comparison view in the UI made it easy to inspect differences between the original and fixed versions, reinforcing user trust in the generated output.

6.3.3 Evaluation Summaries and Risk Assessment

Each review concluded with an evaluation summary produced by the Evaluation Agent. This summary included qualitative explanations of improvements, remaining concerns, and an overall risk-level assessment.

Users were able to quickly understand whether the proposed fix significantly reduced risk and whether additional manual review might still be required. This explicit evaluation step added transparency and helped bridge the gap between automated recommendations and human decision-making.

6.3.4 GitHub Pull Request Experience

In GitHub-based workflows, the qualitative experience closely mirrored real-world development practices. After authentication, users could browse repository files, trigger a review, and generate a pull request directly from the interface.

The automatically created pull requests included:

- A new branch containing the generated fix
- A descriptive PR title and summary derived from agent outputs
- Clean commit history reflecting a single logical change

Screenshots of successful PR creation confirmed that the system not only generated fixes but also integrated smoothly into collaborative version control workflows.

6.4 Quantitative and Observational Results

In addition to qualitative observations, we analyzed quantitative metrics and execution characteristics collected during experimental runs. While the system does

not rely on a fixed benchmark dataset, agent-level metrics and evaluation outputs provide meaningful insight into performance, coverage, and practicality.

6.4.1 Evaluation Scores

Each review session concluded with an evaluation summary generated by the **Evaluation Agent**, which assigned numeric scores across multiple dimensions:

- **Correctness**
- **Security**
- **Code Quality and Style**
- **Overall Risk Level**

Across tested scenarios, the generated fixes consistently achieved **overall scores in the range of 85–90 out of 100**, with security-focused scenarios showing the largest improvements relative to the original code. Risk levels were typically reduced from *medium or high* to *low* after fix generation, indicating effective mitigation of critical issues.

These scores were stored alongside session metadata, enabling historical comparison and future analysis.

6.4.2 Agent Execution Latency

Execution time was measured for each agent as part of the orchestration process and logged via CloudWatch. Observed latency characteristics were as follows:

- **Static Analysis Agent:** sub-second execution due to rule-based checks
- **LLM-backed Agents:** dominant contributors to total latency
- **Total end-to-end review time:** approximately **6–20 seconds**, depending on code length and complexity

Although agents were executed sequentially to maintain deterministic behavior and clear traceability, the observed latency remained acceptable for interactive developer workflows.

6.4.3 Coverage and Detection Effectiveness

From an observational standpoint, the multi-agent design demonstrated strong issue coverage:

- Security vulnerabilities and unsafe patterns were consistently detected by the **Security Agent**
- Logic errors and edge cases were effectively identified by the **Bug Detection Agent**
- Stylistic and maintainability issues were captured by the **Deep Review Agent**
- Redundant or low-priority findings were successfully filtered during aggregation

In most test cases, the combined agent outputs addressed a broad spectrum of issue categories, exceeding what would typically be detected by a single static analysis tool.

6.4.4 System Reliability and Observability

All experimental runs successfully recorded:

- Agent execution timings
- Review session metadata
- Generated artifacts and evaluation summaries

No execution failures were observed during testing, and logs provided sufficient detail to trace the reasoning process of each agent. This level of observability supports debugging, auditing, and iterative system improvement.

7. Evaluation

This section summarizes the system's performance across key evaluation dimensions, focusing on correctness, security, coverage, efficiency, and usability. The evaluation combines automated agent-generated metrics with observational analysis from experimental runs.

7.1 Evaluation Criteria

The system was evaluated using the following criteria:

- **Correctness:** Ability of the Fix Agent to preserve original functionality while resolving identified issues.
- **Security:** Effectiveness of detecting and mitigating common vulnerabilities and unsafe coding practices.
- **Coverage:** Breadth of issue categories detected across multiple agents.
- **Latency:** End-to-end execution time for an interactive review session.
- **MLOps Maturity:** Quality of logging, persistence, and traceability.
- **User Experience:** Ease of use, clarity of feedback, and integration with GitHub workflows.

7.2 Discussion

The evaluation demonstrates that the multi-agent approach provides tangible benefits over single-agent or rule-based systems. By decomposing review tasks into specialized agents, the system achieves higher coverage and clearer explanations while maintaining practical latency.

Security-related findings consistently ranked highest in priority and were effectively addressed in generated fixes. The Evaluation Agent added transparency by explicitly scoring fixes and identifying residual risks, enabling users to make informed decisions before merging changes.

From an engineering perspective, the integration of persistent storage, artifact management, and centralized logging reflects strong MLOps practices and supports reproducibility and auditing.

8. Conclusion

This project presented a cloud-native, AI-powered multi-agent code review system that integrates large language model reasoning with real-world software engineering workflows. By decomposing the code review task into specialized agents, the system achieves comprehensive analysis across correctness, security, and code quality dimensions while maintaining transparency and traceability.

Unlike traditional static analysis tools or single-agent LLM assistants, the proposed system leverages a structured agent orchestration pipeline to generate actionable review comments, produce coherent fixes, and evaluate improvements. The integration of GitHub OAuth and automated pull request creation further demonstrates the system's practicality in collaborative development environments.

Experimental results show that the system consistently identifies security vulnerabilities, logic errors, and maintainability issues, and generates fixes that substantially reduce risk while preserving original intent. Quantitative metrics and observational evidence confirm that the multi-agent design provides strong coverage and acceptable latency for interactive use. Additionally, the incorporation of cloud-based persistence, artifact storage, and centralized logging reflects mature MLOps practices and supports reproducibility and auditability.

Overall, this project demonstrates how agentic AI systems, when combined with thoughtful system design and cloud infrastructure, can meaningfully enhance software engineering workflows. The resulting platform serves as a practical example of applying large language models beyond code generation, positioning AI as an intelligent collaborator in code review and quality assurance.

9. Future Work

While the current system demonstrates the effectiveness of multi-agent LLM-based code review, several extensions can further enhance its capabilities and applicability in real-world software engineering environments.

Multi-File and Repository-Level Analysis

The current implementation focuses on single-file reviews. Future work could extend the agent pipeline to reason over multiple files simultaneously, enabling detection of cross-file dependencies, architectural inconsistencies, and repository-wide patterns.

Repository Graph and Contextual Reasoning

Incorporating repository graphs, call graphs, and dependency metadata would allow agents to reason about broader context, such as how changes in one module affect others. This would improve fix quality and reduce unintended side effects.

Continuous Integration and Test-Aware Agents

Integrating the system with CI pipelines would allow agents to observe test failures, build logs, and runtime errors. Agents could then propose fixes guided by concrete feedback, enabling a tighter feedback loop between code changes and system behavior.

Autonomous GitHub Tooling via MCP

Future versions could integrate Model Context Protocol (MCP)-based GitHub tools, allowing agents to autonomously navigate repositories, open issues, update pull requests, and respond to review feedback in an iterative manner.

Expanded Evaluation and Benchmarking

Introducing standardized benchmarks, regression tracking, and longitudinal evaluation would enable systematic comparison of agent performance over time and across different LLM backends.

On-Device and Cost-Aware Execution

Exploring lightweight or on-device LLM execution, as well as cost-aware agent scheduling, could improve accessibility and efficiency for large-scale adoption.